

aaroncheung.me

A zero-dependency design system built to grow in two directions at once: coding projects and standalone UI/UX properties

Role	Sole designer and developer
Timeline	Originally built in high school; redesigned into its current form over the following years, still actively maintained
Stack	Plain HTML, CSS, and JavaScript, zero frameworks, zero libraries, zero build step
Scope	Main portfolio plus two live linked properties (art, sound), with a third (studies) in progress, all sharing one design system

Where It Actually Started

This wasn't designed from a blank page with a plan. It started in high school as a simple place to host and deploy projects, a name at the top, a short list, a little CSS and JS to dress it up, nothing more ambitious than that. It stayed roughly that shape for a while, getting tuned and tweaked in small ways as I learned more, before I eventually redesigned it into something structurally different from what it had been.

The redesign is the point where this became worth writing about. I wanted a site with room to expand in two directions at once, rather than a single growing list. Coding projects, live demos, technical write-ups, represented one direction. A second direction opened up alongside it: standalone mini-sites, each its own UI/UX and frontend statement, art and sound eventually, and studies now. Those two directions reflect two complementary practices: software development on one side, design and frontend experimentation on the other, and I wanted the site's own structure to hold both without either one crowding out the other.

A Named Influence

The final design pulled significant inspiration from Arthur Wallendorff's software engineering portfolio.¹ I went through his site's code directly while building mine, and the barebones early version of my redesign followed the same rule his does: no dependencies, plain HTML, CSS, and JavaScript only. This is a real influence, not just a vague inspiration.

Zero Dependencies: Accident First, Feature Second

The no-dependency constraint didn't start as a philosophy. It started as an accident, the earliest version of the site simply never happened to pull in a library, and I kept building that way out of habit more than intention. What changed is that partway through, I recognized what that accident had been giving me and decided to keep it on purpose: what began as an accidental constraint became an intentional design principle.

Two things came out of making it deliberate. First, every behavior on the site had to be something I actually understood, since there was no library to defer that understanding to. A problem couldn't be solved by

installing something, it had to be solved by figuring out what was actually going on and writing the fix myself. Second, it kept the site's look distinct almost as a side effect. Frameworks tend to arrive with a visual accent, default animation curves, default spacing scales, default gradients, and skipping all of that meant the site never had to actively fight its way out of looking like every other portfolio built on the same starter template.

A problem couldn't be solved by installing something. It had to be solved by figuring out what was actually going on.

The Architecture: One Design System, Multiple Properties

Shared Architecture

The main portfolio, the art site, the SOUND essay, and now the studies scaffold all share one design system, theme, palette, CRT effect, fonts, and footer chrome, through a small partials system rather than each site maintaining its own copy of shared UI by hand. A page declares where a shared piece of markup belongs with a data-include attribute, and a short loader fetches and swaps in the real HTML at runtime.

Content is JSON-driven rather than hand-edited into markup. Experience, projects, skills, and UI elements each live in their own data file, and a shared renderer merges them into one tree at load time, used for both the left-hand navigation and a separate File Explorer page that presents the same underlying data a different way. Every page node gets registered by a stable id rather than a positional index, so looking up a page doesn't depend on where it happens to sit in the list.

Interactive Systems

The theme system persists across all four properties through consistent localStorage keys, dark and light modes, multiple palettes, font switching, a CRT overlay effect, and reduced-motion support, while still branching per site where it needs to: the art site carries its own separate palette and font namespace rather than forcing every property into visually identical theming.

Smaller utilities demonstrate the same architectural approach at a narrower scale: a status widget that pulls the site's real latest commit from the GitHub API and displays it lazygit-style, and a site-mirror tool that embeds a live, scaled preview of the site so a viewer can flip between desktop and mobile sizing.

The Navigation Problem

Every feature on this site fought the no-dependency constraint to some degree, that was expected. What actually cost the most time wasn't a coding problem at all, it was redesigning the navigation into something a recruiter could understand at a glance without sanding off the site's own character.

I went through several full navigation paradigms before landing on the one that's live now, each with real tradeoffs:

- **A lazygit-style status interface**, taking direct inspiration from Wallendorff's site. It fit the terminal aesthetic naturally but didn't scale well as a way to browse a growing list of distinct pages.
- **A tree structure**. Closer to conventional site navigation, easier to scan, but it lost some of the site's specific character in the process.
- **A boxed structure**. Visually distinct, but it struggled with the same problem as the tree version once there was enough content to actually navigate.

- **A folder structure**, the version live today. A VS Code-style file tree in the nav, readable at a glance, and still visibly a deliberate design choice rather than a generic sidebar.

None of these failed outright, each one worked reasonably well in isolation, which is exactly what made the decision hard. This wasn't a case of ruling out something broken, it was extended A/B testing between several legitimate options, each solving the recruiter-legibility problem a little differently and costing a little different amount of the site's own identity in exchange.

Outcome

The site is live at aaroncheung.me and actively maintained. It currently supports two linked properties, art and sound, beyond the main portfolio, with a third, this studies site, in progress. The main portfolio itself hosts six standalone projects alongside the UI/UX toggles described above.

The architecture pays for itself specifically when the site grows. Adding a new project means dropping a page or folder into the existing projects directory in the nav; the shared renderer and stable-id system pick it up without any restructuring elsewhere. Adding an entire new property is a heavier but still repeatable process: I scaffold a new site with the shared partials and footer chrome already wired in, build and polish it in its own editor window, then fold it back into the main site's navigation and link it from the footer. The two-directions structure it was redesigned around has held up as intended: coding projects and mini-sites have both grown since the redesign without either direction crowding the other out architecturally.

Reflection

What I keep coming back to is that the constraint I'm proudest of wasn't a decision I made on purpose at first. Zero dependencies started as the accidental shape of a high schooler's first attempt at a website, and it only became a real design philosophy once I noticed what it was already doing for me and chose to protect it deliberately. I don't think that makes it less mine, if anything, recognizing a good accident and keeping it on purpose is its own kind of design judgment, just one that shows up after the fact instead of before it.

References

- 1 Wallendorff, A. *Software Engineer* portfolio. wallenart.dev. Direct influence on this project's zero-dependency approach and overall design direction; source code reviewed during development.